

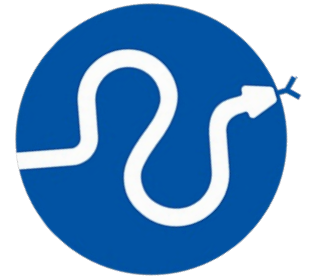
STATS 507

Data Analysis in Python

Week6-1: SciPy and Intro to ML

Dr. Xian Zhang

1. SciPy as Scientific Python
2. A crash course on Learning
(before Scikit learn)



What is SciPy

SciPy: **Sci**entific **Py**thon

Designed and used for scientific and technical computing

SciPy is a library (collection) of **algorithms** and **mathematical** tools built to work with NumPy arrays.

- Mathematics, science, and engineering.
- Can handle operations and data analysis that goes beyond what basic Python or even NumPy can handle.

<https://docs.scipy.org/doc/scipy/tutorial/index.html#user-guide>

When to use SciPy

SciPy is organized into **subpackages** covering different scientific computing domains.

[Special functions](#) (`scipy.special`)

[Integration](#) (`scipy.integrate`)

[Optimization](#) (`scipy.optimize`)

[Interpolation](#) (`scipy.interpolate`)

[Fourier Transforms](#) (`scipy.fft`)

[Signal Processing](#) (`scipy.signal`)

[Linear Algebra](#) (`scipy.linalg`)

[Sparse Arrays](#) (`scipy.sparse`)

[Sparse eigenvalue problems with ARPACK](#)

[Compressed Sparse Graph Routines](#) (`scipy.sparse.csgraph`)

[Spatial data structures and algorithms](#) (`scipy.spatial`)

[Statistics](#) (`scipy.stats`)

[Multidimensional image processing](#) (`scipy.ndimage`)

[File IO](#) (`scipy.io`)

How to use SciPy: importing

Everything in the namespaces of SciPy submodules is **public**. In general in Python, it is recommended to make use of namespaces. For example:

```
my_matrix = np.array([
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
])
# Method 1: Import the entire SciPy package
import scipy
# Usage
eigenvalues = scipy.linalg.eigvals(matrix)
```

```
# Method 2: Import specific submodules
from scipy import special, linalg
# Usage
eigenvalues = linalg.eigvals(matrix)
```

```
# Method 3: Import specific functions
from scipy.linalg import eigvals
# Usage
eigenvalues = eigvals(matrix)
```

```
print(type(eigenvalues))
print(eigenvalues)

<class 'numpy.ndarray'>
[ 1.61168440e+01+0.j -1.11684397e+00+0.j -8.58274334e-16+0.j]
```

SciPy Linear Algebra

There is overlap in the functionality provided by the SciPy and NumPy submodules. However:

- Slightly different from `numpy.linalg`. Always uses BLAS/LAPACK support, which are often highly optimized for specific hardware so could be **faster**.
- Unless you don't want to add SciPy as a dependency to your NumPy program, use `scipy.linalg` instead of `numpy.linalg`

BLAS: basic linear algebra subprogram: <https://www.netlib.org/blas/>

LAPACK: linear algebra Package: <https://www.netlib.org/lapack>

<https://docs.scipy.org/doc/scipy/tutorial/linalg.html>

SciPy Linear Algebra

`scipy.linalg` contains all the functions in `numpy.linalg`

SciPy linear algebra could be **faster**.

<https://docs.scipy.org/doc/scipy/tutorial/linalg.html>

```
import numpy as np
import scipy.linalg as spla
import numpy.linalg as npla
import time
```

Solve a linear system: $Ax = b$

```
# Create a large matrix
```

```
n = 1000
```

```
A = np.random.rand(n, n)
```

```
b = np.random.rand(n)
```

```
# NumPy solve
```

```
start_time = time.time()
```

```
x_np = npla.solve(A, b)
```

```
np_time = time.time() - start_time
```

```
# SciPy solve
```

```
start_time = time.time()
```

```
x_sp = spla.solve(A, b)
```

```
sp_time = time.time() - start_time
```

```
print(f"NumPy solve time: {np_time:.6f} seconds")
```

```
print(f"SciPy solve time: {sp_time:.6f} seconds")
```

```
print(f"Speed difference: {np_time/sp_time:.2f}x")
```

```
NumPy solve time: 0.044730 seconds
```

```
SciPy solve time: 0.020878 seconds
```

```
Speed difference: 2.14x
```

SciPy Sparse

Sparse matrix classes: CSC, CSR, etc.

- functions to **identify** and **build** sparse matrices
- `sparse.linalg` module for sparse linear algebra
- `sparse.csgraph` for sparse graph routines

```
import numpy as np
from scipy import sparse
# 1. Sparse matrix classes: CSC, CSR
dense_matrix = np.array([
    [1, 0, 2],
    [0, 3, 4],
    [5, 6, 0]
])

# Create Compressed Sparse row matrix
csr_matrix = sparse.csr_matrix(dense_matrix)
print("CSR Matrix:")
print(type(csr_matrix))
print(csr_matrix)

CSR Matrix:
<class 'scipy.sparse._csr.csr_matrix'>
(0, 0)    1
(0, 2)    2
(1, 1)    3
(1, 2)    4
(2, 0)    5
(2, 1)    6
```

```
# sparse.linalg module for sparse linear algebra
# Solve a sparse linear system Ax = b
from scipy.sparse import linalg as spla
A = sparse.csr_matrix([[1, 2], [3, 0]])
b = np.array([1, 2])
x = spla.spsolve(A, b)
print("\nSolution to Ax = b:")
print(x)
```

Can be even more efficient
Since we can make use of
the structure

In-class practice

On SciPy Optimization

1. SciPy as Scientific Python
2. A crash course on Learning
(before Scikit learn)

Scikit learn

SciPy Toolkit

Open source machine learning library

- Built atop NumPy, SciPy and Matplotlib.

Makes many common ML/stats models easily available.

- API supports simple modeling fitting, prediction, cross-validation, etc.

https://scikit-learn.org/stable/user_guide.html#user-guide

Section Navigation

1. Supervised learning ✓
2. Unsupervised learning ✓
3. Model selection and evaluation ✓
4. Inspection ✓
5. Visualizations
6. Dataset transformations ✓
7. Dataset loading utilities ✓
8. Computing with scikit-learn ✓
9. Model persistence
10. Common pitfalls and recommended practices
11. Dispatching ✓
12. Choosing the right estimator
13. External Resources, Videos and Talks

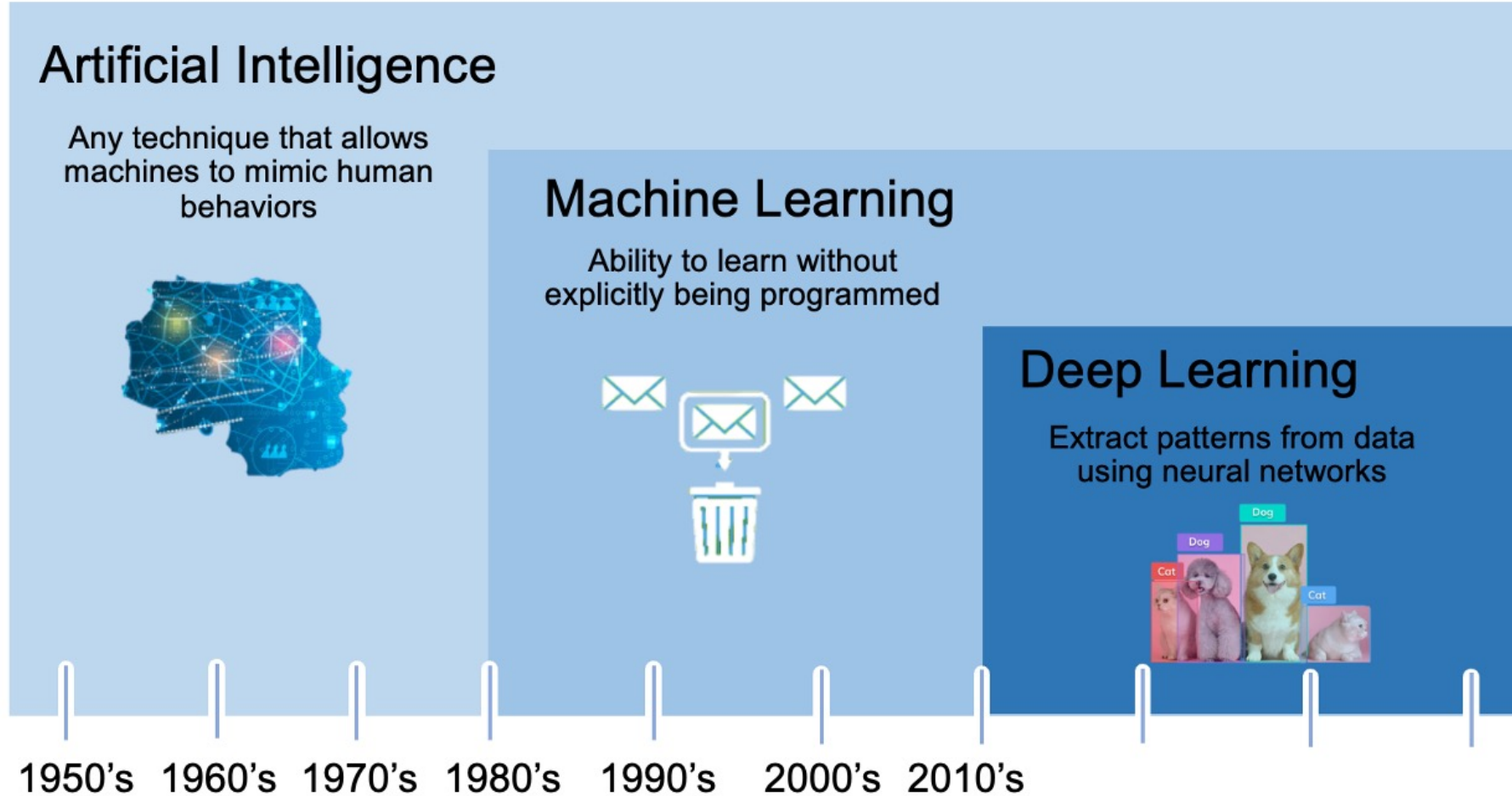


Before the tool...

A crash course on learning.

Ref: <https://udlbook.github.io/udlbook/>

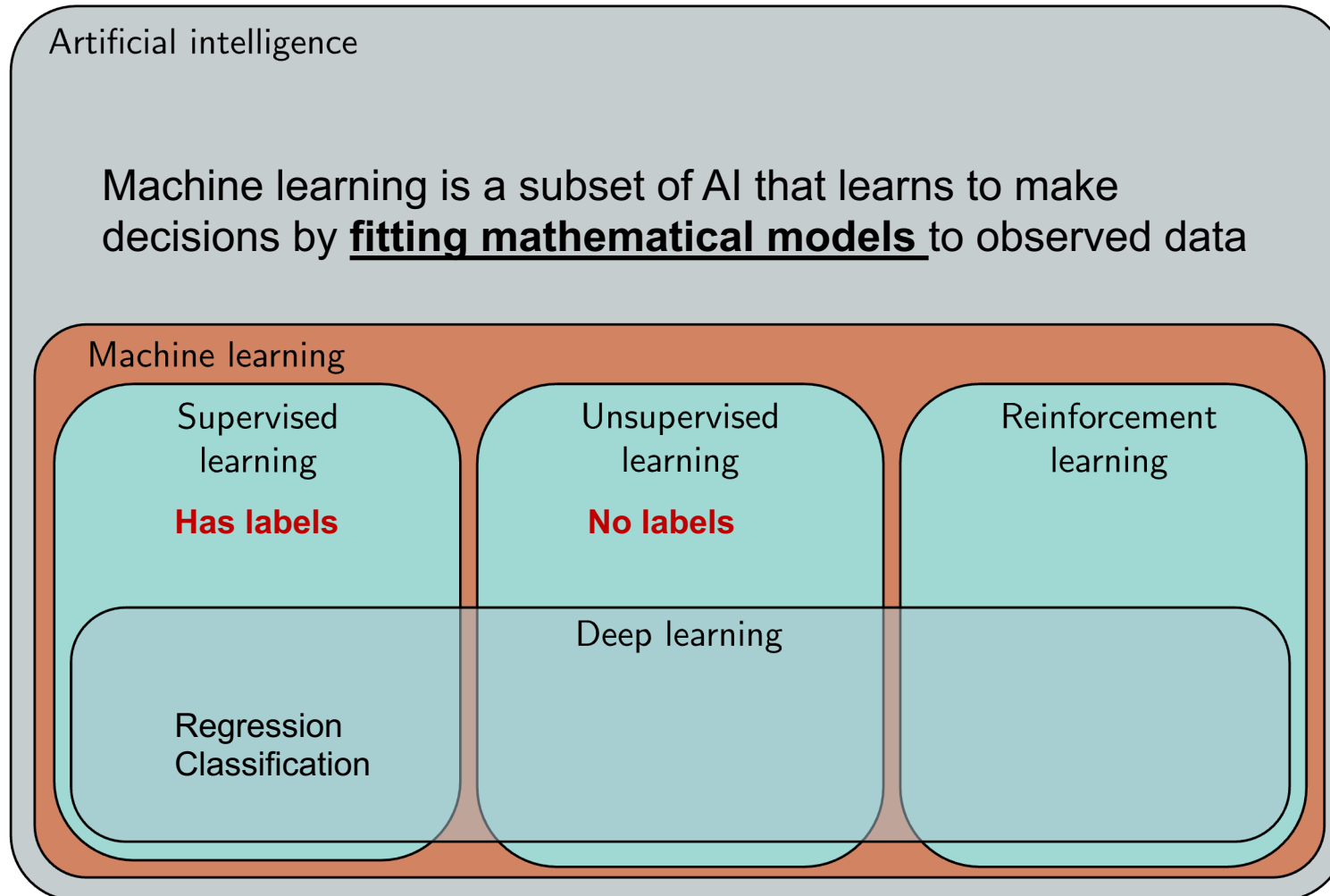
What is Learning?



Slide Credit: Alexander Amini

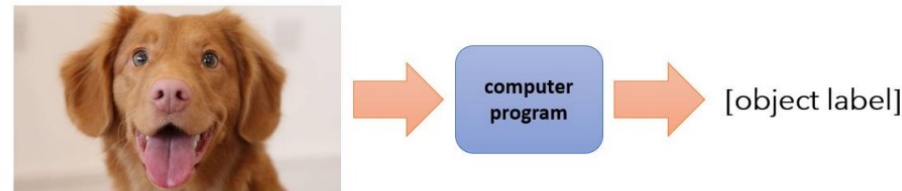
Modified from MIT open course: 6.S191 and Nvidia blog 12

Machine Learning



Supervised Learning

supervised learning



- Classification

input: $\mathbf{x}$

output: $\mathbf{y}$

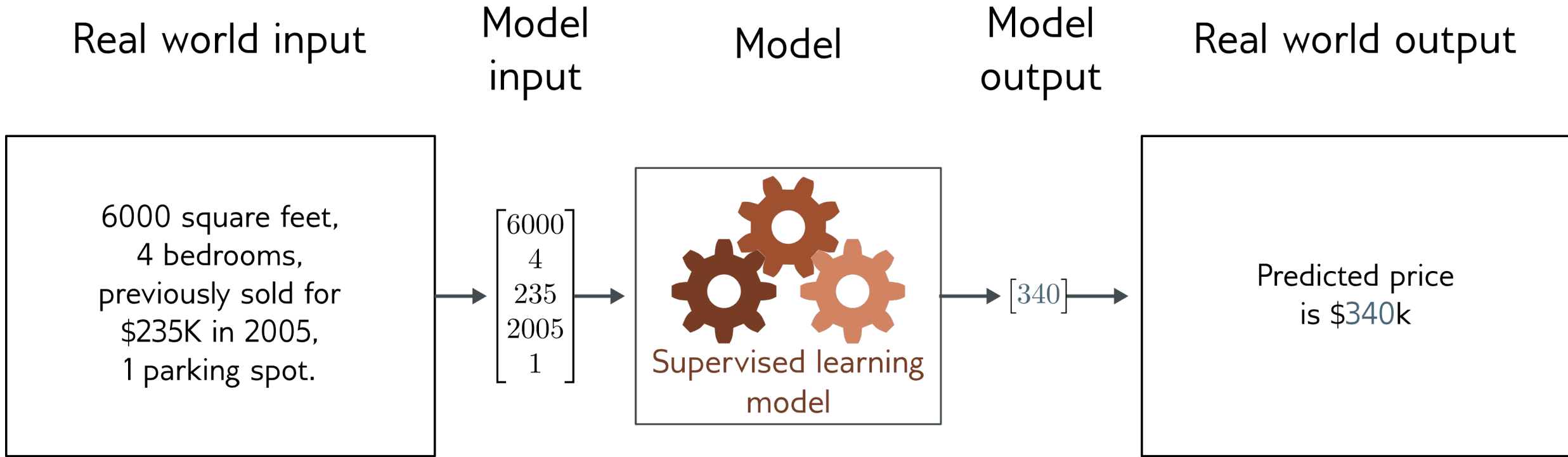
data: $\mathcal{D} = \{(\mathbf{x}_i, \mathbf{y}_i)\}$

goal: $f_{\theta}(\mathbf{x}_i) \approx \mathbf{y}_i$

someone gives
this to you

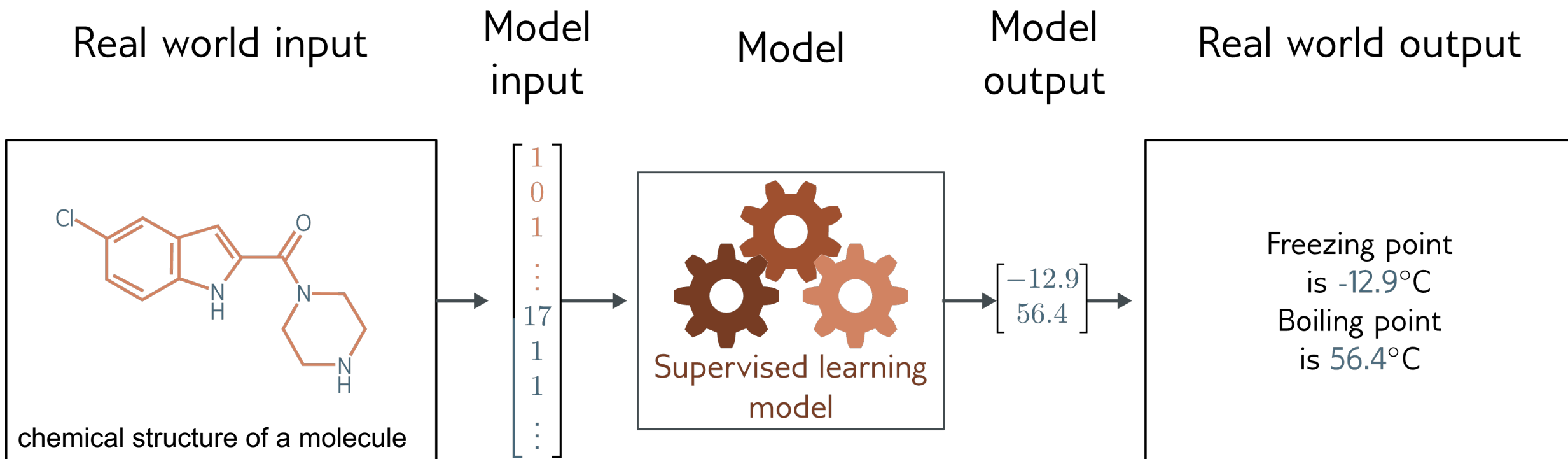
- Define a mapping from input to output
- Learn this mapping from paired input/output data examples.

Univariate Regression problem



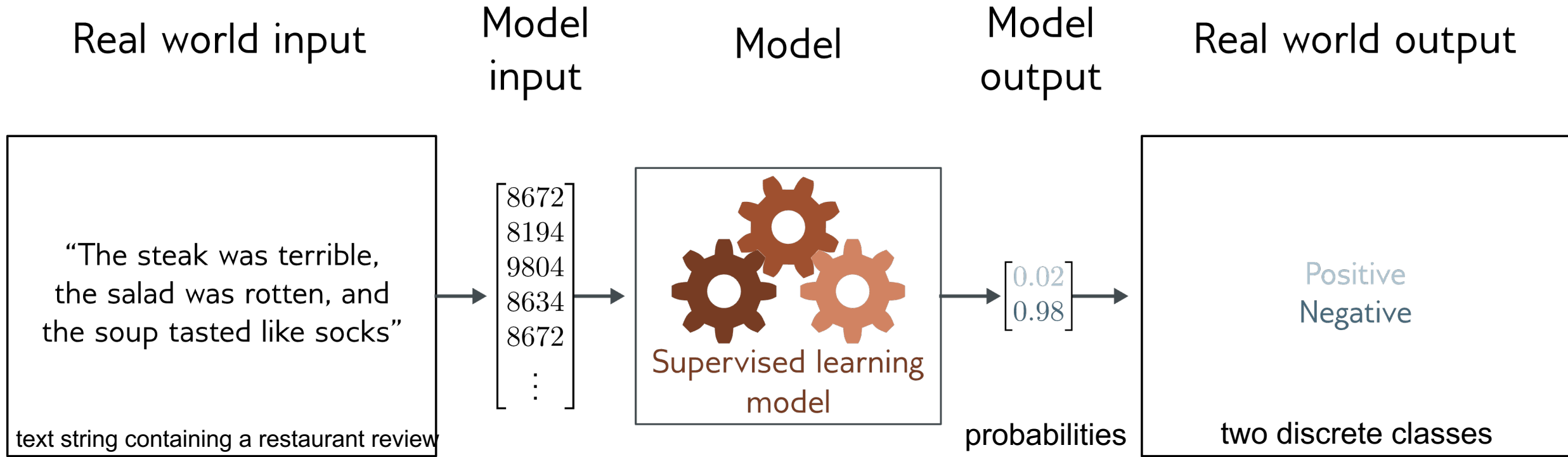
- Univariate regression problem (one output, real value)
- The model: fully connected network

Multivariate Regression problem



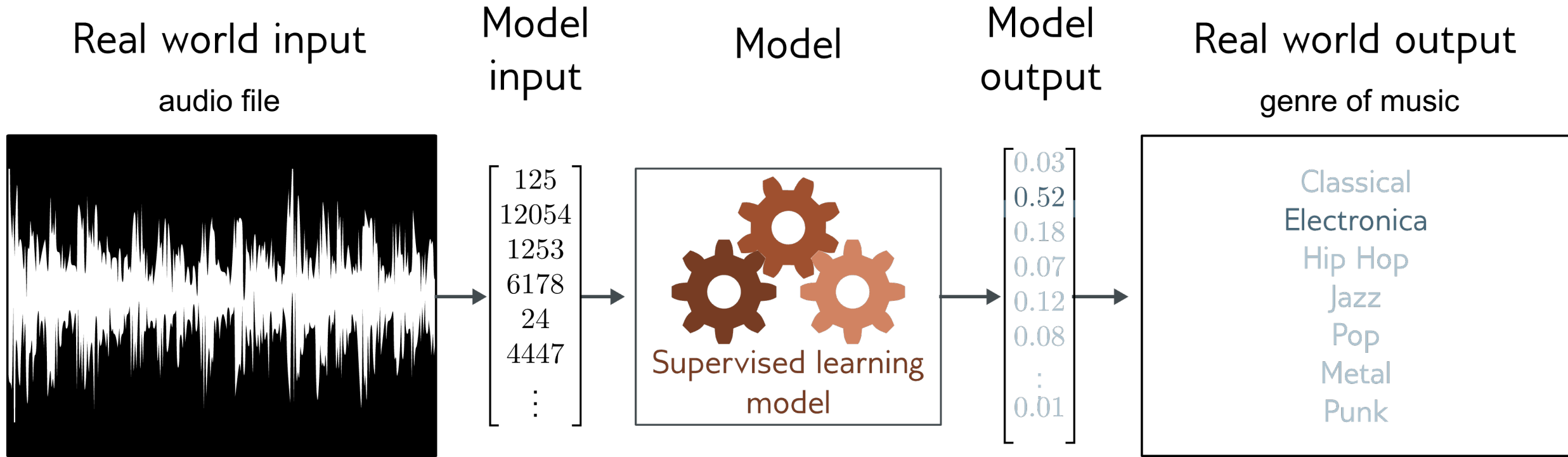
- Multivariate regression problem (>1 output, real value)
- The model: Graph neural network

Binary classification problem



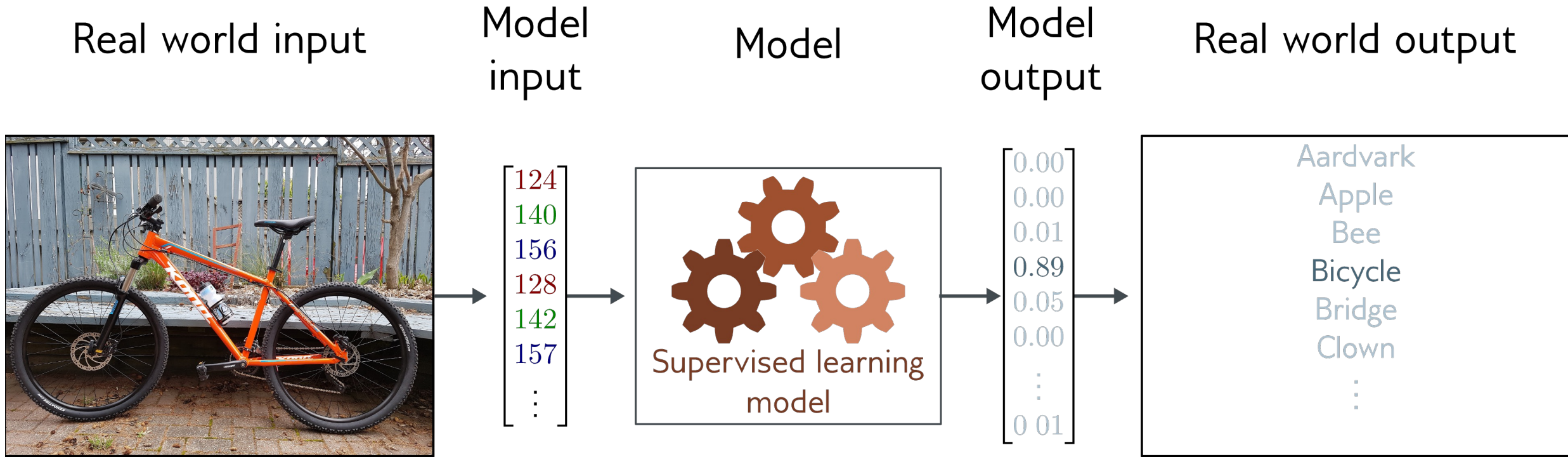
- Binary classification problem (two discrete classes)
- Transformer network

Multi-class classification problem



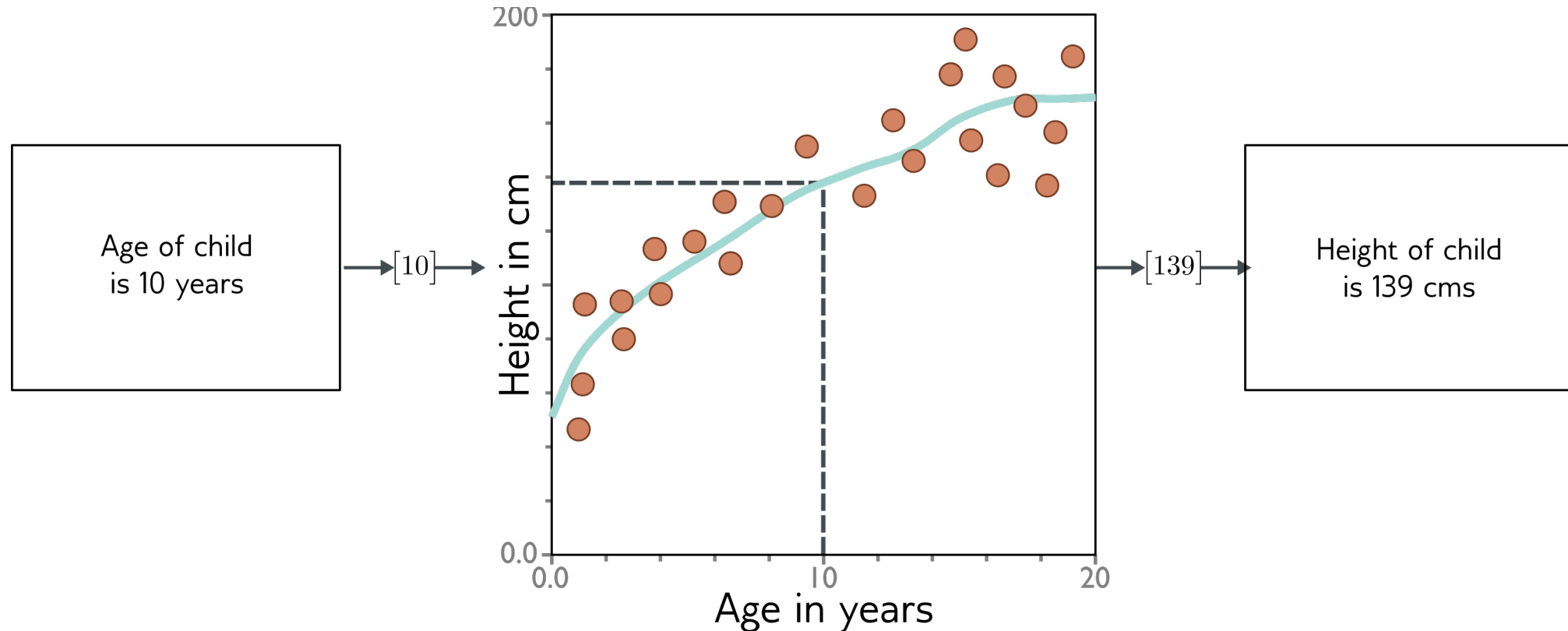
- Multiclass classification problem (discrete classes, >2 possible values)
- Recurrent neural network (RNN)

Multi-class classification problem



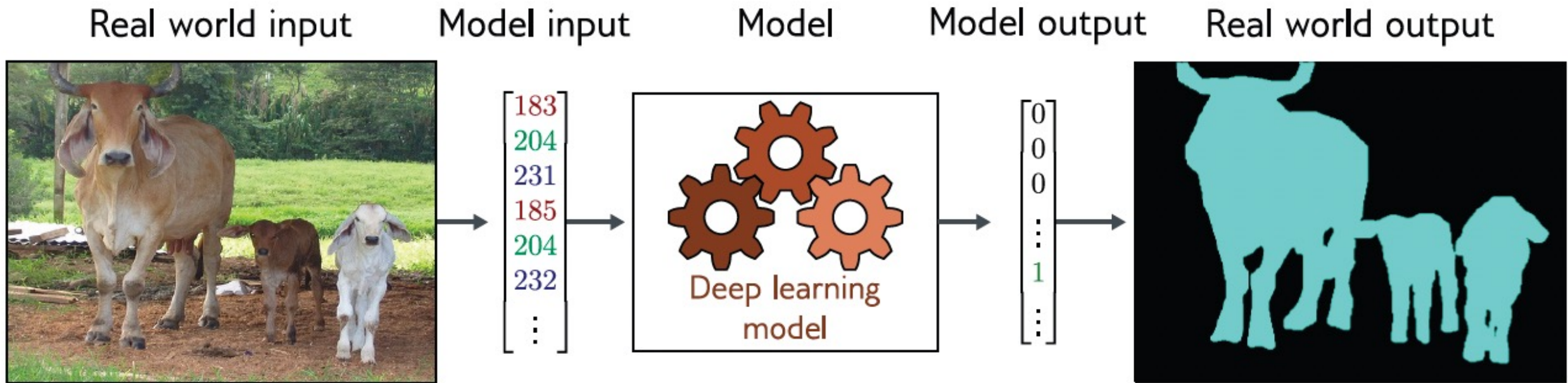
- Multiclass classification problem (discrete classes, >2 possible classes)
- Convolutional network

What exactly is in this black box?



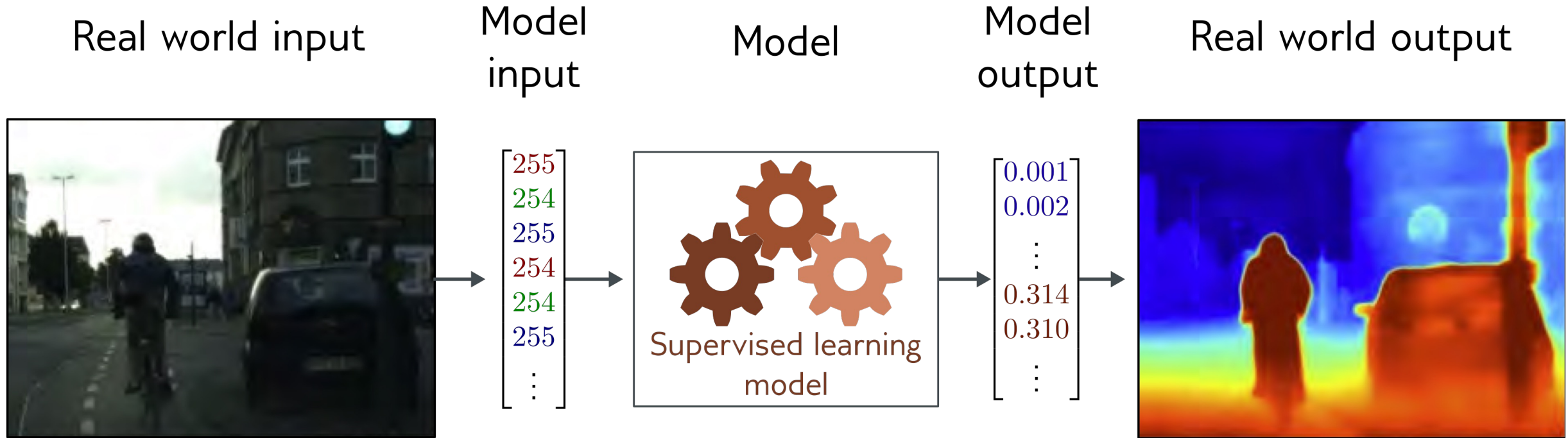
- A family of mathematical equations relating input (age) to output (height)
- Search through family of possible equations to find one that fits training data well

Image segmentation



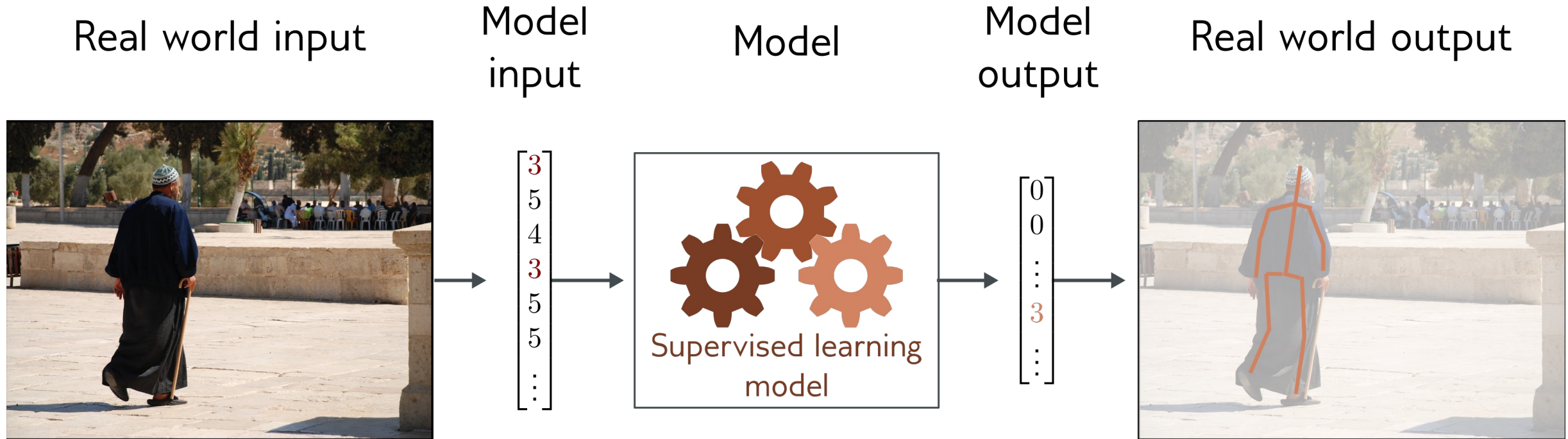
- Multivariate binary classification problem (many outputs, two discrete classes)
- Convolutional encoder-decoder network

Depth Estimation



- Multivariate regression problem (many outputs, continuous)
- Convolutional encoder-decoder network

Pose estimation



- Multivariate regression problem (many outputs, continuous)
- Convolutional encoder-decoder network

Terms for supervised problems

- Regression = continuous numbers as output
- Classification = discrete classes as output
- Two class and multiclass classification treated differently
- Univariate = one output
- Multivariate = more than one output

Some more difficult tasks...

Translation

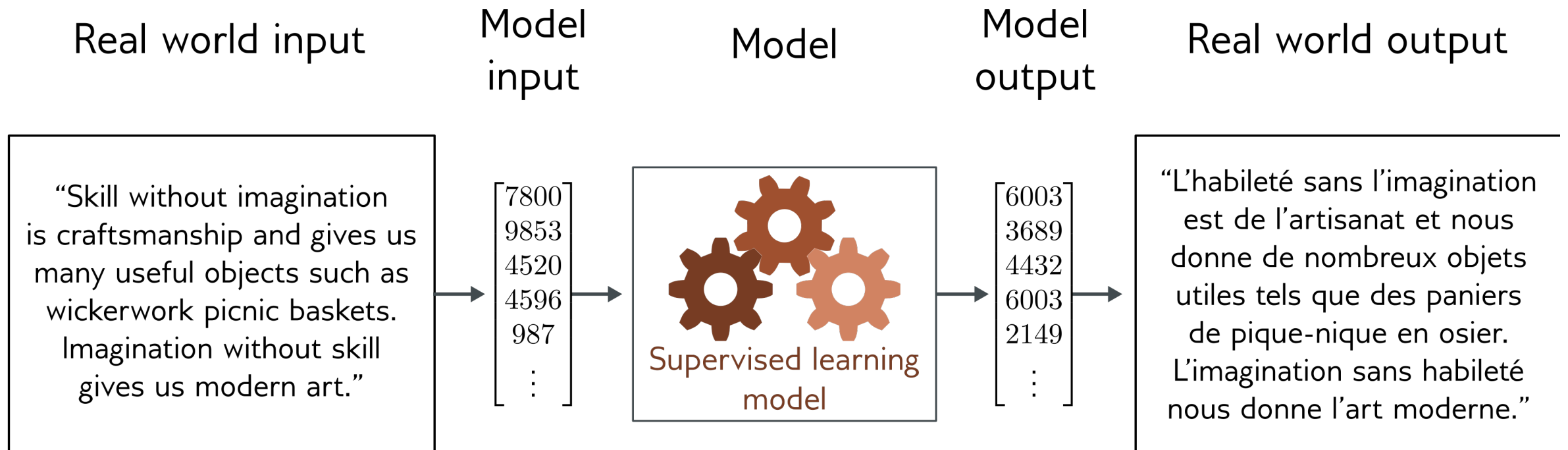


Image caption

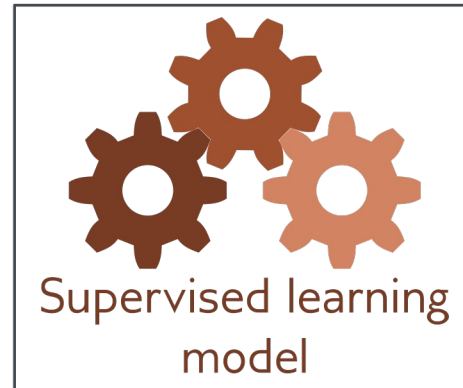
Real world input



Model
input

$$\begin{bmatrix} 183 \\ 204 \\ 231 \\ 185 \\ 204 \\ 232 \\ \vdots \end{bmatrix}$$

Model



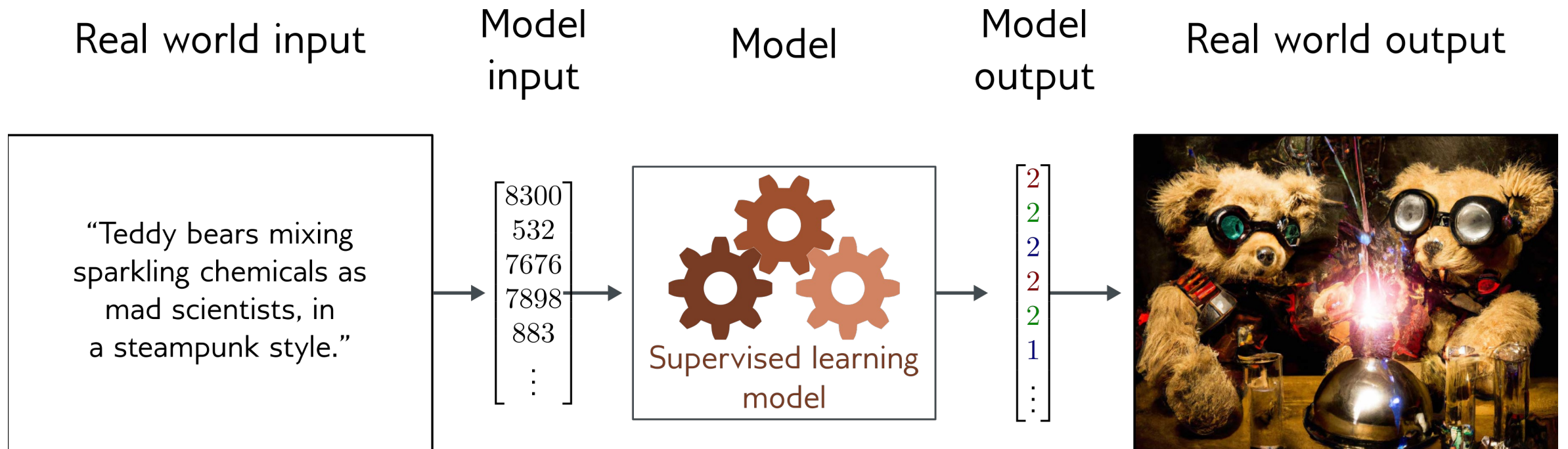
Model
output

$$\begin{bmatrix} 1 \\ 5593 \\ 7532 \\ 7924 \\ 1 \\ \vdots \end{bmatrix}$$

Real world output

"A Kazakh man on a
horse holding a
bird of prey"

Image generation from text



What do those examples have in common?

- Very complex relationship between input and output
- Sometimes may be **many** possible valid answers
- But outputs (and sometimes inputs) obey rules

“A Kazakh man on a
horse holding a
bird of prey”

Language obeys
grammatical rules



Natural images also
have “rules”

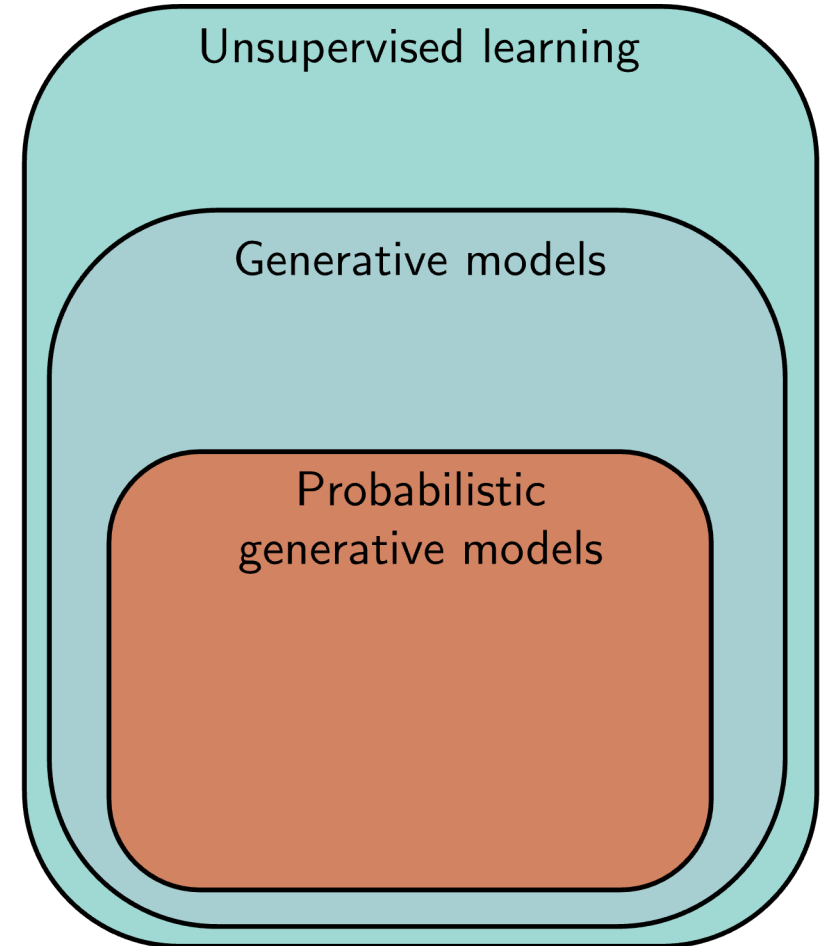
Idea

- Learn the “grammar” of the data from unlabeled examples
- Can use a gargantuan amount of data to do this (as unlabeled)
- Make the supervised learning task easier by having a lot of knowledge of possible outputs

Unsupervised Learning

Unsupervised Learning

- Learning about a dataset **without labels**
 - e.g., clustering
- Generative models can **create examples**
 - e.g., generative adversarial networks
- PGMs **learn distribution** over data
 - e.g., variational autoencoders,
 - e.g., normalizing flows,
 - e.g., diffusion models



Generative Models



Generate images

ChatGPT 4o ▾

write a poem about Michigan Fall



Michigan Fall

Golden leaves in whispered flight,
A tapestry of autumn light,
Lake's cool breath on every shore,
As trees ignite in crimson lore.

Maple's fire, oak's embrace,
A symphony of nature's grace,
Cider's warmth, the harvest sweet,
Crunching paths beneath our feet.

Winds that hum through orchard rows,
As apples blush in sunset glows,
The air is crisp, the skies are clear,
October sings the end of year.

Generate texts

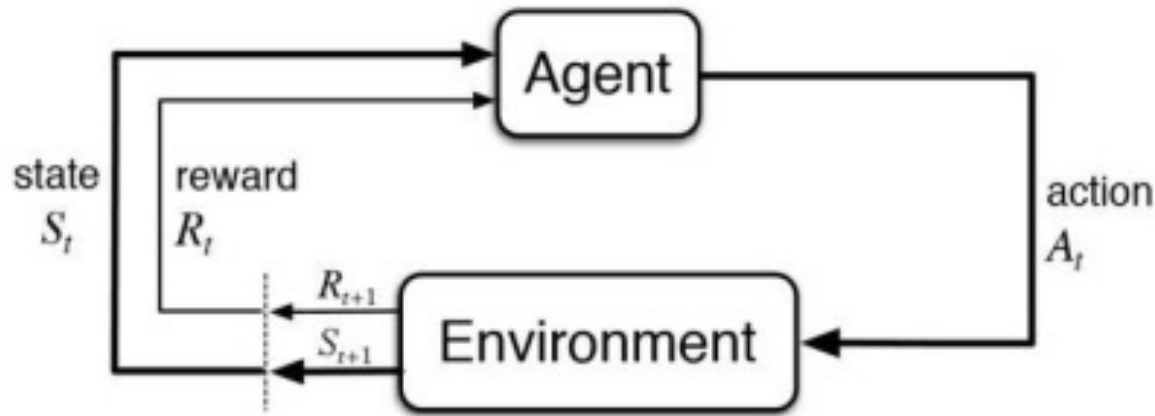
Reinforcement learning

Ref: <https://udlbook.github.io/udlbook/>

<https://rail.eecs.berkeley.edu/deeprlcourse/>

Reinforcement learning

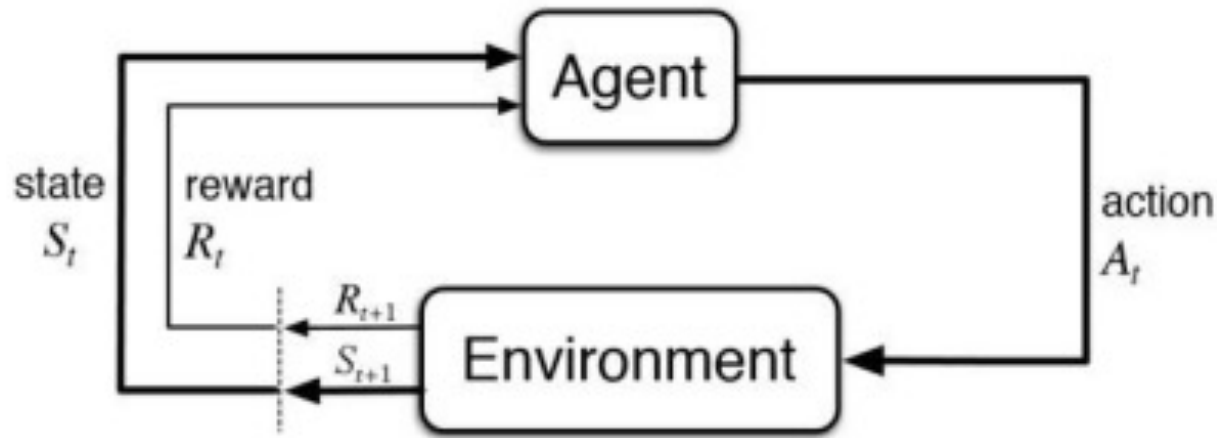
Goal: **Agent** take actions (decision making) to change the state so that you receive rewards from the environment.



<https://www.nature.com/articles/nature16961>

How is this different from other machine learning topics?

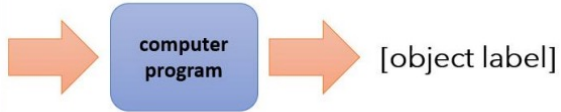
Reinforcement learning



- Data is not **i.i.d.** (independent and identically distributed): previous outputs influence future inputs.
- Ground truth answer is not known, only know we succeeded or failed. More generally, we only know the **reward**.
- Sometimes agents don't receive any data – agents have to explore the environment to **gather data** as you go.

Reinforcement learning

supervised learning



input: $\mathbf{x}$

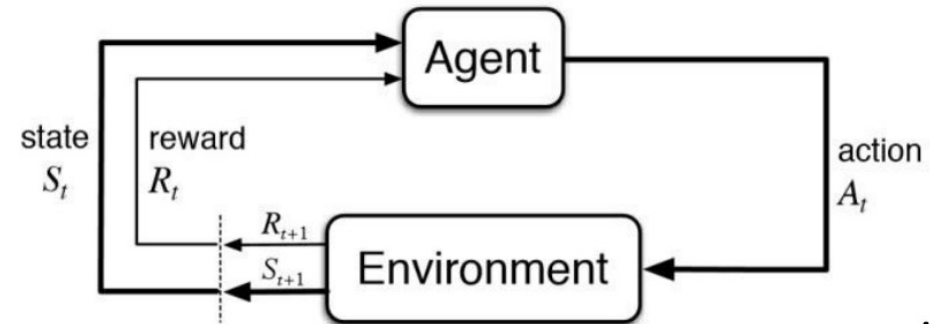
output: $\mathbf{y}$

data: $\mathcal{D} = \{(\mathbf{x}_i, \mathbf{y}_i)\}$

goal: $f_{\theta}(\mathbf{x}_i) \approx \mathbf{y}_i$

← someone gives
this to you

reinforcement learning



input: $\mathbf{s}_t$ at each time step

output: $\mathbf{a}_t$ at each time step

data: $(\mathbf{s}_1, \mathbf{a}_1, r_1, \dots, \mathbf{s}_T, \mathbf{a}_T, r_T)$

goal: learn $\pi_{\theta} : \mathbf{s}_t \rightarrow \mathbf{a}_t$

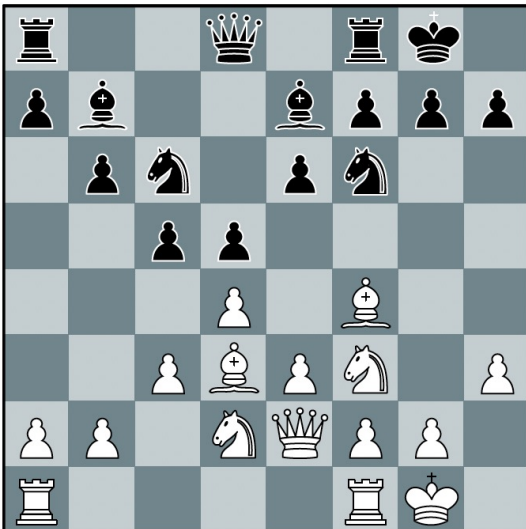
to maximize $\sum_t r_t$

pick your
own actions

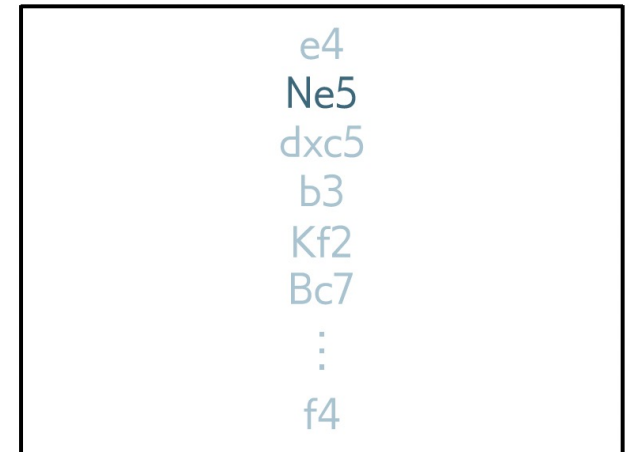
Example: chess

- States are valid states of the chess board
- Actions at a given time are valid possible moves
- Positive rewards for taking pieces, negative rewards for losing them

State

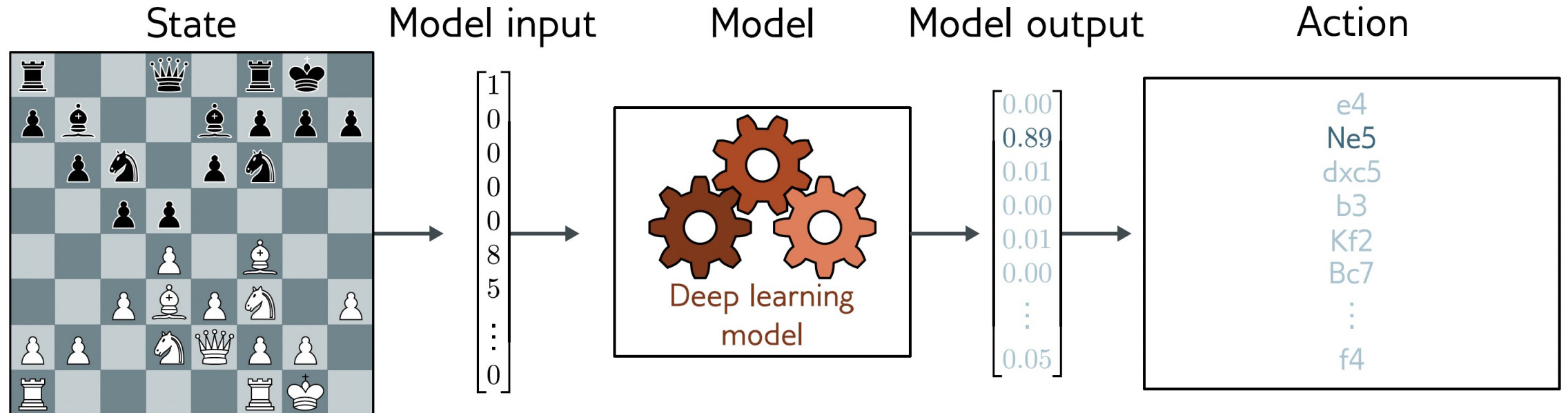


: Action



Example: chess

- States are valid states of the chess board
- Actions at a given time are valid possible moves
- Positive rewards for taking pieces, negative rewards for losing them



2018 Turing award winners



The Nobel Prize in Physics 2024



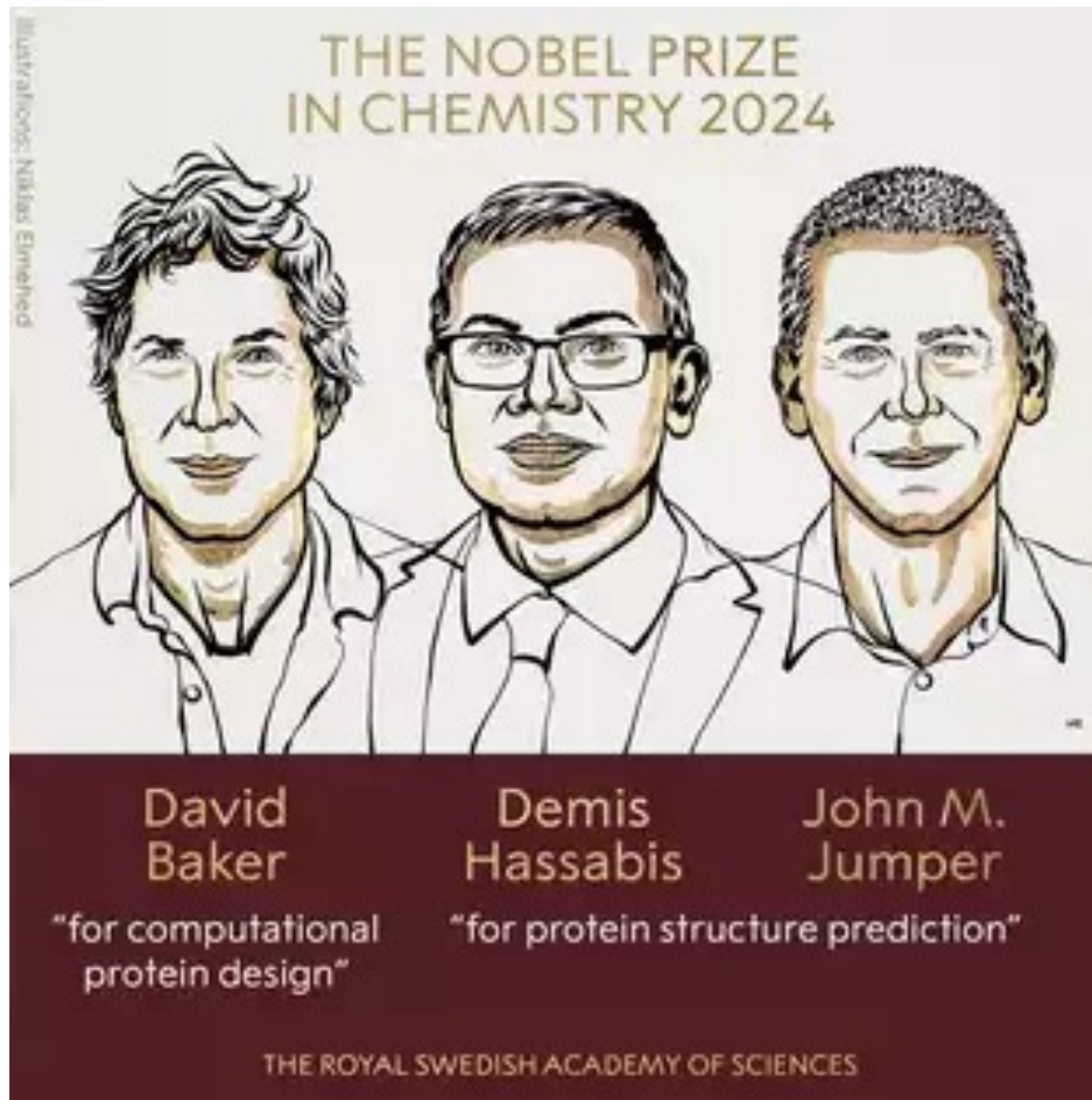
Ill. Niklas Elmehed © Nobel Prize
Outreach
John J. Hopfield
Prize share: 1/2



Ill. Niklas Elmehed © Nobel Prize
Outreach
Geoffrey E. Hinton
Prize share: 1/2

The Nobel Prize in Physics 2024 was awarded jointly to John J. Hopfield and Geoffrey E. Hinton "for foundational discoveries and inventions that enable machine learning with artificial neural networks"

<https://www.nobelprize.org/prizes/physics/2024/summary/>



“Demis Hassabis and John Jumper have developed an AI model to solve a 50-year-old problem: predicting proteins’ complex structures.”

<https://www.nobelprize.org/prizes/chemistry/2024/summary/>

Coming next: scikit-learn for learning problems



The **MNIST database** (Modified [National Institute of Standards and Technology](#) database) is a large [database](#) of handwritten digits

Other things

HW5 due this week.

HW6 out on Thursday.

Midterm Accommodations.